

# AWS SNS and SQS Integration Project

## Project Overview

This section details the step-by-step process for integrating Amazon Simple Notification Service (SNS) with Amazon Simple Queue Service (SQS). The goal was to establish a publish-subscribe messaging workflow, allowing SNS to fan out messages to SQS queues for asynchronous processing.

### Step 1: SNS Topic Creation

The integration process began with the creation of an SNS topic. Using the SNS Management Console, a Standard topic type was selected and named `content_topic`. The display name assigned was "Content topic." The topic was created successfully, establishing the foundation for sending notifications.

The screenshot shows the 'Create topic' page in the AWS SNS console. The 'Details' section is active, showing two options for topic type: 'FIFO (first-in, first-out)' and 'Standard'. The 'Standard' option is selected. Below the type selection, there is a 'Name' field containing 'content\_topic' and a 'Display name - optional' field containing 'Content topic'. The console also shows a breadcrumb trail at the bottom: 'content\_topic' > 'Standard' > 'arn:aws:sns'.

**Create topic**

**Details**

Type [Info](#)  
Topic type cannot be modified after topic is created

☐ FIFO (first-in, first-out)

- Strictly-preserved message ordering
- Exactly-once message delivery
- Subscription protocols: SQS

☒ Standard

- Best-effort message ordering
- At-least once message delivery
- Subscription protocols: SQS, Lambda, Data Firehose, HTTP, SMS, email, mobile application endpoints

**Name**

content\_topic

Maximum 256 characters. Can include alphanumeric characters, hyphens (-) and underscores (\_).

**Display name - optional** [Info](#)  
To use this topic with SMS subscriptions, enter a display name. Only the first 10 characters are displayed in an SMS message.

Content topic

Maximum 100 characters.

[content\\_topic](#) Standard arn:aws:sns

### Step 2: SQS Queue Setup

Next, an SQS queue was created through the SQS Management Console. A Standard queue type was chosen, and the queue was named `content_q`. Several settings were configured:

- Visibility timeout: 10 seconds

- Message retention period: 5 days (note: although the referenced PDF suggested 15 minutes, 5 days was used)
- Maximum message size: 64 KB

### Details

**Type**  
Choose the queue type for your application or cloud infrastructure.

☒ **Standard** [Info](#)  
At-least-once delivery, message ordering isn't preserved
 

- At-least once delivery
- Best-effort ordering

☐ **FIFO** [Info](#)  
First-in-first-out delivery, message ordering is preserved
 

- First-in-first-out delivery
- Exactly-once processing

**Name**

A queue name is case-sensitive and can have up to 80 characters. You can use alphanumeric characters, hyphens (-), and underscores (\_).

### Configuration

Set the maximum message size, visibility to other consumers, and message retention.

**Visibility timeout** [Info](#)

 Seconds

Should be between 0 seconds and 12 hours.

**Message retention period** [Info](#)

 Days

Should be between 1 minute and 14 days.

**Delivery delay** [Info](#)

 Seconds

Should be between 0 seconds and 15 minutes.

**Maximum message size** [Info](#)

 KiB

Should be between 1 KiB and 1024 KiB.

**Receive message wait time** [Info](#)

|                       |                           |          |                        |   |   |                          |
|-----------------------|---------------------------|----------|------------------------|---|---|--------------------------|
| <input type="radio"/> | <a href="#">content_q</a> | Standard | 2025-12-22T16:07:06:00 | 0 | 0 | Amazon SQS key (SSE-SQS) |
|-----------------------|---------------------------|----------|------------------------|---|---|--------------------------|

## Step 3: Queue Verification

After creation, the queue `content_q` appeared in the list of available queues. It was confirmed to have been created on 12/22/2025 at 4:07 PM (UTC-6). At this point, there were zero messages available in the queue. The queue was encrypted using the Amazon SQS key (SSE-SQS).

## Step 4: Establishing the SNS to SQS Subscription

To connect the SNS topic to the SQS queue, a subscription was created. Returning to the SNS `content_topic`, it was noted that there were no existing subscriptions. A new subscription was initiated by selecting the Amazon SQS protocol and choosing the ARN of the `content_q` queue as the endpoint.

## Create subscription

### Details

#### Topic ARN

arn:aws:sns:us-east-1:123456789012:content\_topic

#### Protocol

The type of endpoint to subscribe

Amazon SQS

#### Endpoint

Only Amazon SQS standard queues will be listed and can receive notifications from an Amazon SNS standard topic.

arn:aws:sqs:us-east-1:123456789012:MyQueue

☐ Enable raw message delivery

## Step 5: Publishing a Test Message

A test message was published to verify the integration. The subject of the message was "sample message," with the body containing "this is a sample message." A message attribute was added with the following details:

- Type: String.Array (the PDF indicated String, but String.Array was used)
- Name: attr1
- Value: value1

The message was then published to the SNS topic.

### Message details

#### Topic ARN

arn:aws:sns:us-east-1:123456789012:content\_topic

#### Subject - optional

sample message

Maximum 100 printable ASCII characters

### Message attributes [Info](#)

Message attributes let you provide structured metadata items (such as timestamps, geospatial data, signatures, and identifiers) for the message.

| Type                                   | Name  | Value  |                         |
|----------------------------------------|-------|--------|-------------------------|
| String.Array                           | attr1 | value1 | <button>Remove</button> |
| <button>Add another attribute</button> |       |        |                         |

### Message body to send to the endpoint

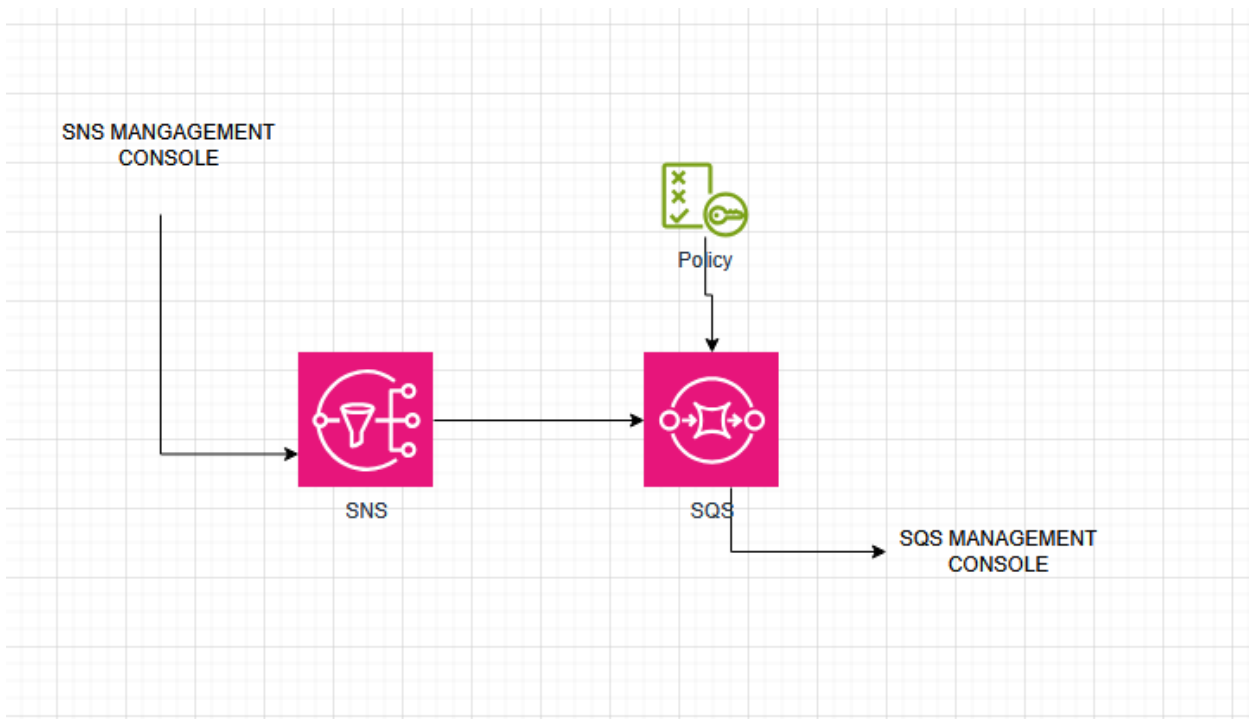
1 this is a sample message

## Step 6: Message Delivery Verification

The final step involved confirming that the message was successfully delivered. Upon checking the SQS console, the `content_q` queue now showed one message available. This verified that the SNS topic had successfully delivered the message to the SQS queue.

|  | Name                      | Type     | Created                | Messages available |
|--|---------------------------|----------|------------------------|--------------------|
|  | <a href="#">content_q</a> | Standard | 2025-12-22T16:07:06:00 | 1                  |

Architecture:



## Summary

The AWS SNS/SQS integration project was completed successfully. The message flow worked as intended:

SNS Topic (content\_topic) → Subscription → SQS Queue (content\_q)

This project demonstrated the publish-subscribe messaging pattern, showcasing how SNS can distribute messages to SQS queues for asynchronous processing.

## References:

### 1. Amazon Simple Notification Service (SNS) Documentation

- URL: <https://docs.aws.amazon.com/sns/>

- Description: Official AWS documentation covering SNS topics, subscriptions, message publishing, and protocols. Essential for understanding how to create and manage SNS topics.

### 2. Amazon Simple Queue Service (SQS) Documentation

- URL: <https://docs.aws.amazon.com/sqs/>

- Description: Complete guide to SQS queue types, message handling, visibility timeout, retention periods, and queue configuration. Critical for understanding queue behavior and settings.

### 3. Fanout to Amazon SQS Queues - SNS Tutorial

- URL: <https://docs.aws.amazon.com/sns/latest/dg/sns-sqs-as-subscriber.html>

- Description: Step-by-step guide on subscribing SQS queues to SNS topics, including IAM policy requirements and best practices for the fanout pattern used in this project.

#### 4. Amazon SQS Access Policy Examples

- URL:

<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-creating-custom-policies-access-policy-examples.html>

- Description: Documentation on SQS access policies including examples for granting SNS permission to send messages to queues. Explains the JSON policy structure used in this project.